



US 20250278672A1

(19) **United States**(12) **Patent Application Publication**
LING et al.(10) **Pub. No.: US 2025/0278672 A1**(43) **Pub. Date: Sep. 4, 2025**(54) **SYSTEMS AND METHODS FOR
OPTIMIZING HYPERPARAMETERS OF
MACHINE LEARNING MODELS USING
REDUCTION ITERATION TECHNIQUES**(52) **U.S. Cl.**
CPC **G06N 20/00** (2019.01)(71) Applicant: **TATA CONSULTANCY SERVICES
LIMITED**, Mumbai (IN)(57) **ABSTRACT**(72) Inventors: **YIBEI LING**, Edison, NJ (US);
CHARLES NJELITA, Edison, NJ
(US)(73) Assignee: **TATA CONSULTANCY SERVICES
LIMITED**, Mumbai (IN)(21) Appl. No.: **19/062,417**(22) Filed: **Feb. 25, 2025**(30) **Foreign Application Priority Data**

Feb. 29, 2024 (IN) 202421015031

Publication Classification(51) **Int. Cl.**
G06N 20/00 (2019.01)

The most fundamental task in ML models is to automate the setting of hyperparameters to optimize performance. Traditionally, in machine learning (ML) models hyperparameter optimization problem has been solved using brute-force techniques such as grid search, and the like. This strategy exponentially increases computation costs and memory overhead. Considering the complexity and variety of the ML models there still remains practical difficulties of selecting right combinations of hyperparameters to maximize performance of the ML models. Embodiments of the present disclosure provide systems and methods for hyperparameters optimization in machine learning models and to effectively reduce the hyperparameter search dimensions and identify the important hyperparameter dimensions that are high variable to identify the best hyperparameter thereby saving the computing energy of machine learning process and eliminate categorical dimensions by using a combination of reduction-iteration techniques.

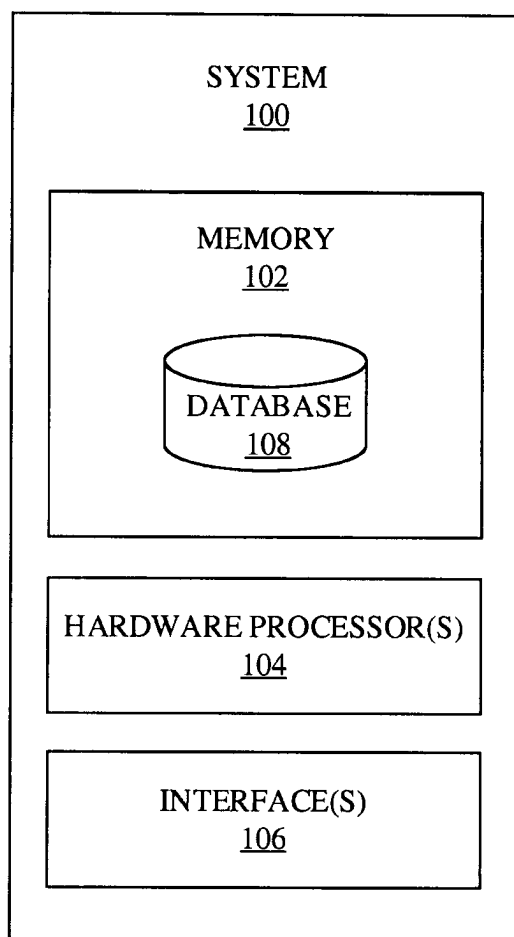


FIG. 1

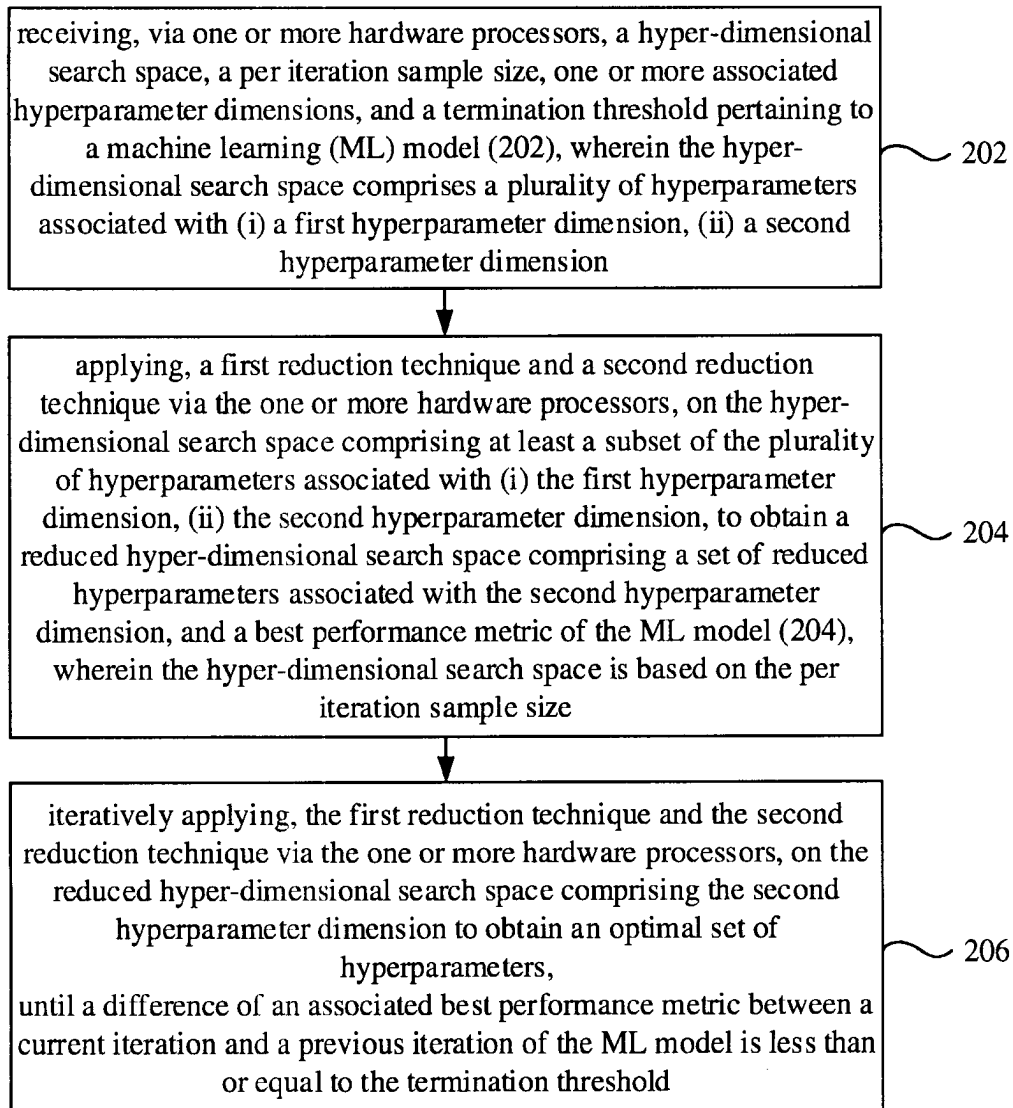


FIG. 2

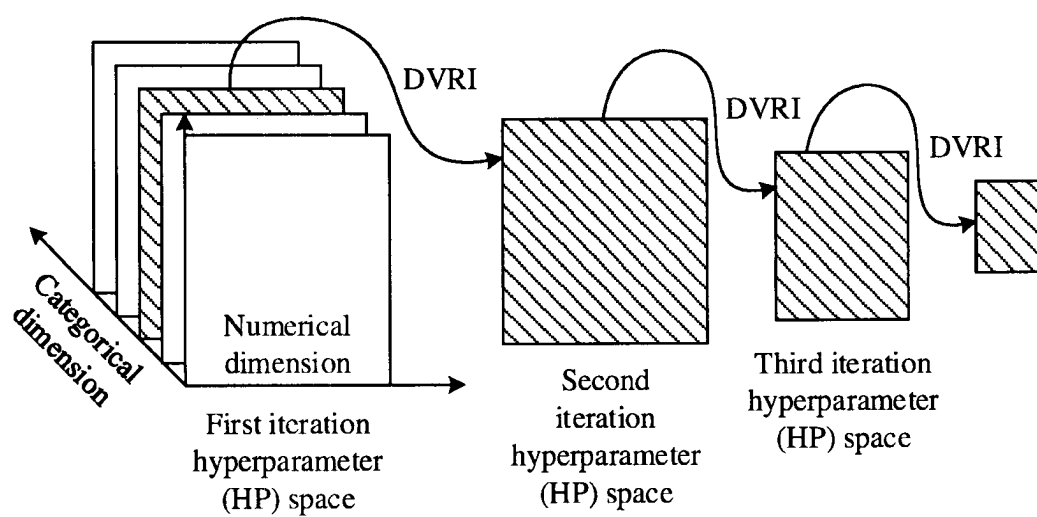


FIG. 3

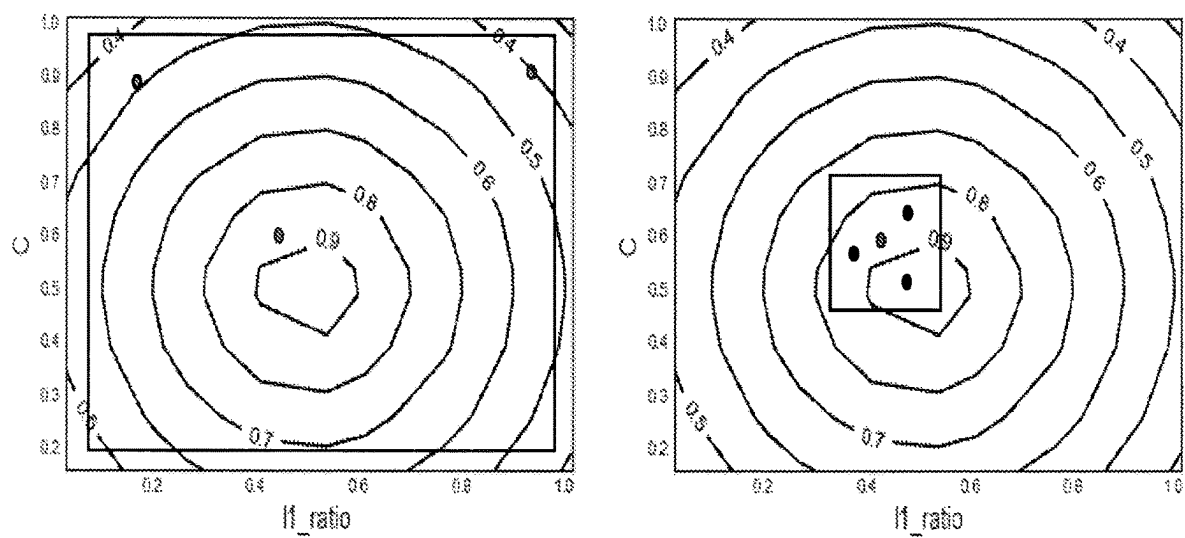
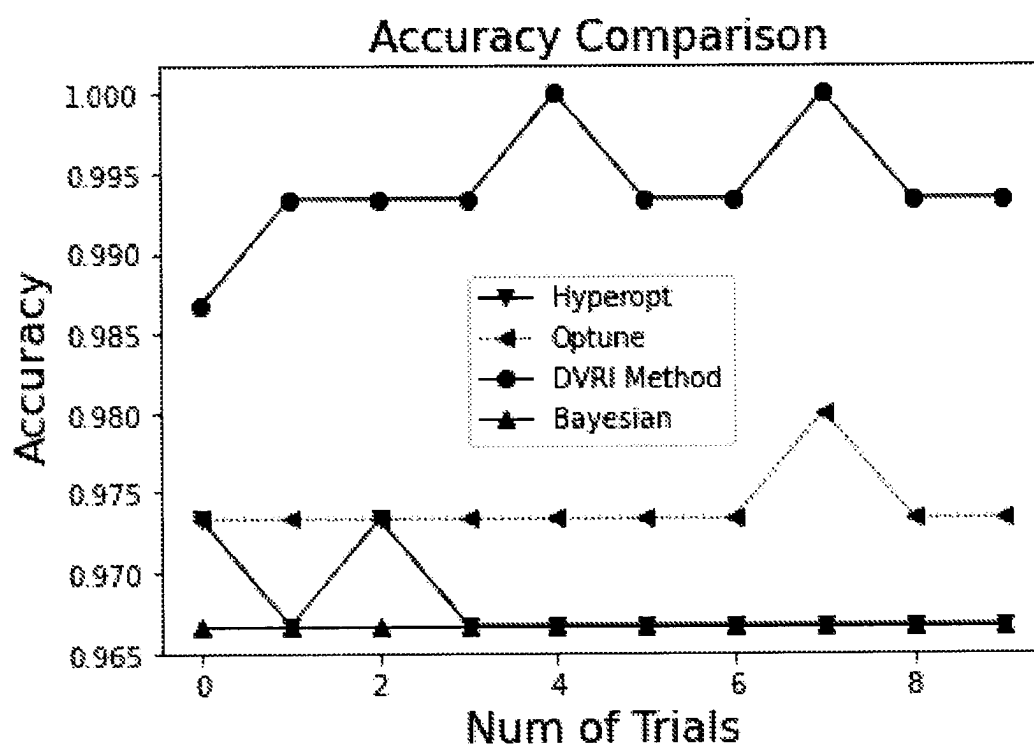
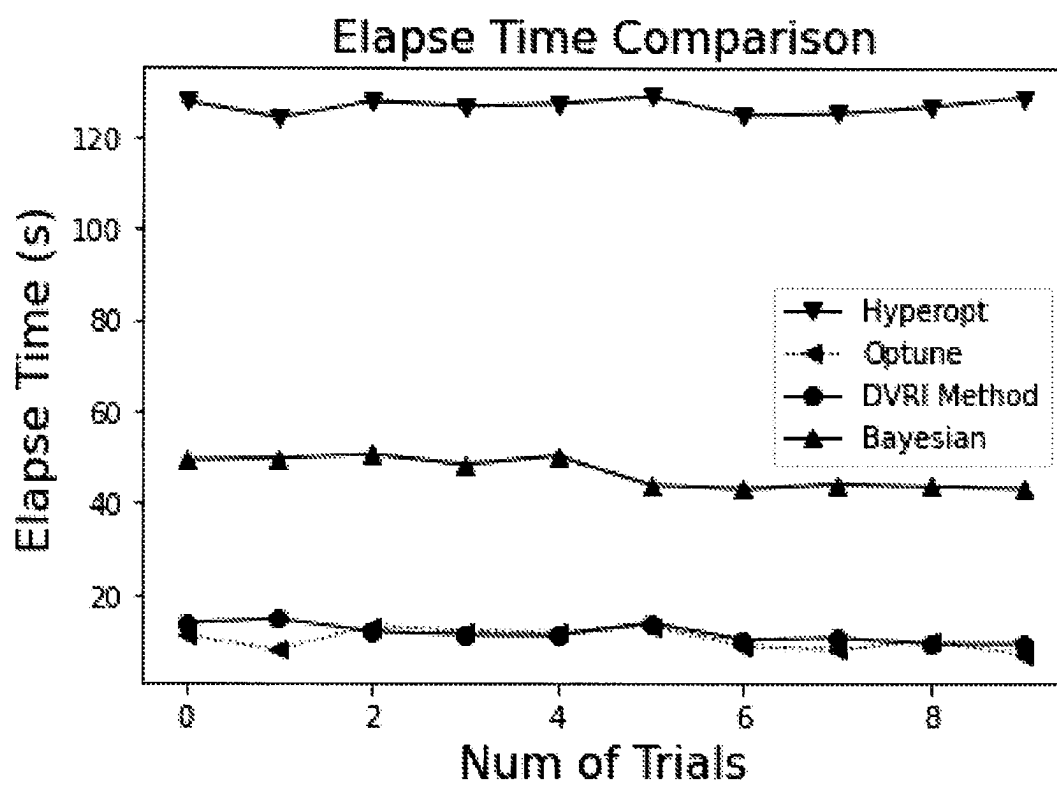


FIG. 4

**FIG. 5A**

**FIG. 5B**

SYSTEMS AND METHODS FOR OPTIMIZING HYPERPARAMETERS OF MACHINE LEARNING MODELS USING REDUCTION ITERATION TECHNIQUES

PRIORITY CLAIM

[0001] This U.S. patent application claims priority under 35 U.S.C. § 119 to: Indian Patent Application number 202421015031, filed on Feb. 29, 2024. The entire contents of the aforementioned application are incorporated herein by reference.

TECHNICAL FIELD

[0002] The disclosure herein generally relates to optimization techniques, and, more particularly, to systems and methods for optimizing hyperparameters of machine learning models using reduction iteration techniques.

BACKGROUND

[0003] Machine learning (ML) models have been widely used in several applications and domains. This is due to their inherent generic nature and ability to demonstrate high performance. To fit a machine learning model into various problems, the hyperparameters must be tuned. Hyperparameter tuning process differs based on the ML model type, and associated hyperparameters.

[0004] The most fundamental task in ML models is to automate the setting of hyperparameters to optimize performance. Traditionally, in machine learning (ML) models hyperparameter optimization problem has been solved using brute-force techniques such as grid search, and the like. This strategy exponentially increases computation costs and memory overhead. To overcome the above challenges, Bayesian Optimization (BO) techniques were implemented while assuming a certain prior distribution over the cost function. Though the BO techniques are popular for its implementation, these are not stable in providing needed performance. Yet another traditional approach has been manual testing and tuning of hyperparameters. However, manual approach is ineffective for many problems due to various factors including iteration size that undergoes sampling during optimization process. Considering the complexity and variety of the ML models there still remains practical difficulties of selecting right combinations of hyperparameters to maximize performance of the ML models.

SUMMARY

[0005] Embodiments of the present disclosure present technological improvements as solutions to one or more of the above-mentioned technical problems recognized by the inventors in conventional systems. For example, in one aspect, there is provided a processor implemented method for optimizing hyperparameters of machine learning models using reduction iteration techniques. The method comprises receiving, via one or more hardware processors, a hyper-dimensional search space, a per iteration sample size, one or more associated hyperparameter dimensions, and a termination threshold pertaining to a machine learning (ML) model, wherein the hyper-dimensional search space comprises a plurality of hyperparameters associated with (i) a first hyperparameter dimension, and (ii) a second hyperparameter dimension; applying, a first reduction technique and

a second reduction technique via the one or more hardware processors, on the hyper-dimensional search space comprising at least a subset of the plurality of hyperparameters associated with (i) the first hyperparameter dimension, (ii) the second hyperparameter dimension, to obtain a reduced hyper-dimensional search space comprising a set of reduced hyperparameters associated with the second hyperparameter dimension, and a best performance metric of the ML model, wherein the hyper-dimensional search space is based on the per iteration sample size; and iteratively applying, the first reduction technique and the second reduction technique via the one or more hardware processors, on the reduced hyper-dimensional search space comprising the second hyperparameter dimension to obtain an optimal set of hyperparameters, until a difference of an associated best performance metric between a current iteration and a previous iteration of the ML model is less than or equal to the termination threshold.

[0006] In an embodiment, shrinkage of the hyper-dimensional search space between the current iteration and the previous iteration is at the rate of (x/y) a per-iteration, where d is a dimensionality of the second hyperparameter dimension, and x and y are integer values.

[0007] In an embodiment, the first reduction technique is a Dimension-Reduction Iteration (DRI) technique.

[0008] In an embodiment, the second reduction technique is a Volume-Reduction Iteration (VRI) technique.

[0009] In an embodiment, the first reduction technique and the second reduction technique are concurrently applied during each iteration.

[0010] In an embodiment, the first hyperparameter dimension is a categorical hyperparameter dimension, and the second hyperparameter dimension is a numerical hyperparameter dimension.

[0011] In another aspect, there is provided a processor implemented system for optimizing hyperparameters of machine learning models using reduction iteration techniques. The system comprises: a memory storing instructions; one or more communication interfaces; and one or more hardware processors coupled to the memory via the one or more communication interfaces, wherein the one or more hardware processors are configured by the instructions to: receive a hyper-dimensional search space, a per iteration sample size, one or more associated hyperparameter dimensions, and a termination threshold pertaining to a machine learning (ML) model, wherein the hyper-dimensional search space comprises a plurality of hyperparameters associated with (i) a first hyperparameter dimension, and (ii) a second hyperparameter dimension; apply, a first reduction technique and a second reduction technique, on the hyper-dimensional search space comprising at least a subset of the plurality of hyperparameters associated with (i) the first hyperparameter dimension, (ii) the second hyperparameter dimension, to obtain a reduced hyper-dimensional search space comprising a set of reduced hyperparameters associated with the second hyperparameter dimension, and a best performance metric of the ML model, wherein the hyper-dimensional search space is based on the per iteration sample size; and iteratively apply, the first reduction technique and the second reduction technique, on the reduced hyper-dimensional search space comprising the second hyperparameter dimension to obtain an optimal set of hyperparameters, until a difference of an associated best performance metric between

a current iteration and a previous iteration of the ML model is less than or equal to the termination threshold.

[0012] In an embodiment, shrinkage of the hyper-dimensional search space between the current iteration and the previous iteration is at the rate of $(x/y)^d$ per-iteration, where d is a dimensionality of the second hyperparameter dimension, and x and y are integer values.

[0013] In an embodiment, the first reduction technique is a Dimension-Reduction Iteration (DRI) technique.

[0014] In an embodiment, the second reduction technique is a Volume-Reduction Iteration (VRI) technique.

[0015] In an embodiment, the first reduction technique and the second reduction technique are concurrently applied during each iteration.

[0016] In an embodiment, the first hyperparameter dimension is a categorical hyperparameter dimension, and the second hyperparameter dimension is a numerical hyperparameter dimension.

[0017] In yet another aspect, there are provided one or more non-transitory machine-readable information storage mediums comprising one or more instructions which when executed by one or more hardware processors cause optimizing hyperparameters of machine learning models using reduction iteration techniques by receiving a hyper-dimensional search space, a per iteration sample size, one or more associated hyperparameter dimensions, and a termination threshold pertaining to a machine learning (ML) model, wherein the hyper-dimensional search space comprises a plurality of hyperparameters associated with (i) a first hyperparameter dimension, and (ii) a second hyperparameter dimension; applying, a first reduction technique and a second reduction technique, on the hyper-dimensional search space comprising at least a subset of the plurality of hyperparameters associated with (i) the first hyperparameter dimension, (ii) the second hyperparameter dimension, to obtain a reduced hyper-dimensional search space comprising a set of reduced hyperparameters associated with the second hyperparameter dimension, and a best performance metric of the ML model, wherein the hyper-dimensional search space is based on the per iteration sample size; and iteratively applying, the first reduction technique and the second reduction technique, on the reduced hyper-dimensional search space comprising the second hyperparameter dimension to obtain an optimal set of hyperparameters, until a difference of an associated best performance metric between a current iteration and a previous iteration of the ML model is less than or equal to the termination threshold.

[0018] In an embodiment, shrinkage of the hyper-dimensional search space between the current iteration and the previous iteration is at the rate of $(1/4)^d$ per-iteration, where d is a dimensionality of the second hyperparameter dimension, and x and y are integer values.

[0019] In an embodiment, the first reduction technique is a Dimension-Reduction Iteration (DRI) technique.

[0020] In an embodiment, the second reduction technique is a Volume-Reduction Iteration (VRI) technique.

[0021] In an embodiment, the first reduction technique and the second reduction technique are concurrently applied during each iteration.

[0022] In an embodiment, the first hyperparameter dimension is a categorical hyperparameter dimension, and the second hyperparameter dimension is a numerical hyperparameter dimension.

[0023] It is to be understood that both the foregoing general description and the following detailed description are exemplary and explanatory only and are not restrictive of the invention, as claimed.

BRIEF DESCRIPTION OF THE DRAWINGS

[0024] The accompanying drawings, which are incorporated in and

[0025] constitute a part of this disclosure, illustrate exemplary embodiments and, together with the description, serve to explain the disclosed principles:

[0026] FIG. 1 depicts an exemplary system for optimizing hyperparameters of machine learning models using reduction iteration techniques, in accordance with an embodiment of the present disclosure.

[0027] FIG. 2 depicts an exemplary flow chart illustrating a method for optimizing hyperparameters of machine learning models using reduction iteration techniques, using the system of FIG. 1, in accordance with an embodiment of the present disclosure.

[0028] FIG. 3 depicts per-iteration volume shrinkage upon application of the first reduction technique and the second reduction technique on the hyperparameter space, in accordance with an embodiment of the present disclosure.

[0029] FIG. 4 depicts a graphical representation illustrating samples taking place in two consecutive iterations (left-side figure: dots representing the sampled points in the hyperparameter space, right-side figure: black dots representing the sampled points in the subsequent iteration), in accordance with an embodiment of the present disclosure.

[0030] FIGS. 5A and 5B depict a graphical representation illustrating performance comparison the method of FIG. 2 and conventional approaches (e.g., Hyperopt®, Optune®, Bayesian optimization, etc.), in accordance with an embodiment of the present disclosure.

DETAILED DESCRIPTION

[0031] Exemplary embodiments are described with reference to the accompanying drawings. In the figures, the left-most digit(s) of a reference number identifies the figure in which the reference number first appears. Wherever convenient, the same reference numbers are used throughout the drawings to refer to the same or like parts. While examples and features of disclosed principles are described herein, modifications, adaptations, and other implementations are possible without departing from the scope of the disclosed embodiments.

[0032] The most fundamental task in ML models is to automate the setting of hyperparameters to optimize performance. Traditionally, in machine learning (ML) models hyperparameter optimization problem has been solved using brute-force techniques such as grid search, and the like. This strategy exponentially increases computation costs and memory overhead. To overcome the above challenges, Bayesian Optimization (BO) techniques were implemented while assuming a certain prior distribution over the cost function. Though the BO techniques are popular for its implementation, these are not stable in providing needed performance. Yet another traditional approach has been manual testing and tuning of hyperparameters. However, manual approach is ineffective for many problems due to various factors including iteration size that undergoes sampling during optimization process. Considering the com-

plexity and variety of the ML models there still remains practical difficulties of selecting right combinations of hyperparameters to maximize performance of the ML models.

[0033] Embodiments of the present disclosure provide systems and methods for hyperparameters optimization in machine learning. More specifically, the system and method make use of small random samples to effectively reduce the hyperparameter search dimensions and identify the important hyperparameter dimensions that are high variable to the best hyperparameter. The system then performs randomized search on the reduced important dimensions. As a result, the method of the present disclosure can efficiently identify the best hyperparameters, thereby saving the computing energy of machine learning process. The method of randomized grid search is to take random samples over the entire specified hyperparameter spaces (on the global scope). If the hyperparameter space is large, the system cannot find the best hyper-parameter with small sample size. However, if sample size is very large, then it incurs a substantial computing resource in the search. As such the randomized grid search is ineffective in handling large hyperparameter space. To overcome the above challenges, the system and method of the present invention take a small number of samples on the high dimensional hyperparameter space to explore the location of the best optimal hyperparameter combination, and then makes use of the best hyperparameters in the sampled data to construct smaller hyper-parameter space and compares the best performance in the sampled data with the previous one. The process is performed in an iterative fashion, until the performance gain in two consecutive steps is less than a threshold.

[0034] More specifically, a dimension-volume-reduction iteration (DVRI) hyperparameter optimizer has been incorporated an auto-ML framework which plays an important role in reducing computational overhead and energy consumption as a byproduct.

[0035] The hyper-parameter optimization presents two primary challenges: Firstly, the hyper-parameter space, comprising mixed data types like categorical, float, and integer, is non-differentiable. Secondly, the hyper-parameter optimization is inherently non-convex. Consequently, randomized grid search is employed as an effective and robust means for hyperparameter optimization. The main objective of randomized grid search is to generate random combinations in the hyper-parameter space and select the best hyper-parameter combination. However, a key limitation of randomized hyper-parameter optimization is the need 1) to define the hyperparameter space where search initiates and 2) to determine a proper sample size for randomized search.

[0036] The dimensionality of hyperparameter space varies with models, can be as high as m-dimensional space. A small sample size often leads to sub-optimal or inferior performance for large and high-dimensional hyper-parameter space, whereas a large sample size consumes excessive computational resources and power consumption.

[0037] The uniqueness of this dimension and volume-reduction iteration method stems from a crucial observation: categorical hyper-parameters show significant variability in terms of their impact on the model's performance. Capitalizing on this characteristic, the method in each iteration employs small random samples to pinpoint the optimal values for these hyper-parameter dimensions. After identifying these values, these categorical dimensions are

excluded from further exploration. For numerical hyper-parameters (integers and floats), the method similarly uses small random samples to locate potential areas where values for each dimension may exist. This technique effectively reduces the search space at a ratio of $(1/4)^d$ at each iteration, where d represents the dimensionality of the numerical hyper-parameters. A combination of dimension-reduction and volume-reduction iteration techniques turns out to be extremely efficient to identify the optimal hyper-parameter combination.

[0038] Referring now to the drawings, and more particularly to FIG. 1 through 5B, where similar reference characters denote corresponding features consistently throughout the figures, there are shown preferred embodiments and these embodiments are described in the context of the following exemplary system and/or method.

[0039] FIG. 1 depicts an exemplary system 100 for optimizing hyperparameters of machine learning models using reduction iteration techniques, in accordance with an embodiment of the present disclosure. In an embodiment, the system 100 includes one or more hardware processors 104, communication interface device(s) or input/output (I/O) interface(s) 106 (also referred as interface(s)), and one or more data storage devices or memory 102 operatively coupled to the one or more hardware processors 104. The one or more processors 104 may be one or more software processing components and/or hardware processors. In an embodiment, the hardware processors can be implemented as one or more microprocessors, microcomputers, microcontrollers, digital signal processors, central processing units, state machines, logic circuitries, and/or any devices that manipulate signals based on operational instructions. Among other capabilities, the processor(s) is/are configured to fetch and execute computer-readable instructions stored in the memory. In an embodiment, the system 100 can be implemented in a variety of computing systems, such as laptop computers, notebooks, hand-held devices (e.g., smartphones, tablet phones, mobile communication devices, and the like), workstations, mainframe computers, servers, a network cloud, and the like.

[0040] The I/O interface device(s) 106 can include a variety of software and hardware interfaces, for example, a web interface, a graphical user interface, and the like and can facilitate multiple communications within a wide variety of networks N/W and protocol types, including wired networks, for example, LAN, cable, etc., and wireless networks, such as WLAN, cellular, or satellite. In an embodiment, the I/O interface device(s) can include one or more ports for connecting a number of devices to one another or to another server.

[0041] The memory 102 may include any computer-readable medium known in the art including, for example, volatile memory, such as static random-access memory (SRAM) and dynamic-random access memory (DRAM), and/or non-volatile memory, such as read only memory (ROM), erasable programmable ROM, flash memories, hard disks, optical disks, and magnetic tapes. In an embodiment, a database 108 is comprised in the memory 102, wherein the database 108 comprises information pertaining to machine learning model(s), associated hyper-dimensional search space, a per iteration sample size, one or more associated hyperparameter dimensions, and a termination threshold, and the like. The database 108 further comprises reduced hyper-dimensional search space comprising reduced hyper-

parameters set associated with a specific hyperparameter dimension, and the like. The memory **102** stores various techniques such as a first reduction technique and a second reduction technique which when executed by the system **100** enable to perform the method described herein. The memory **102** further comprises (or may further comprise) information pertaining to input(s)/output(s) of each step performed by the systems and methods of the present disclosure. In other words, input(s) fed at each step and output(s) generated at each step are comprised in the memory **102** and can be utilized in further processing and analysis.

[0042] FIG. 2, with reference to FIG. 1, depicts an exemplary flow chart illustrating a method for optimizing hyperparameters of machine learning models using reduction iteration techniques, using the system **100** of FIG. 1, in accordance with an embodiment of the present disclosure. In an embodiment, the system(s) **100** comprises one or more data storage devices or the memory **102** operatively coupled to the one or more hardware processors **104** and is configured to store instructions for execution of steps of the method by the one or more processors **104**. The steps of the method of the present disclosure will now be explained with reference to components of the system **100** of FIG. 1, and the flow diagram as depicted in FIG. 2. Although process steps, method steps, techniques or the like may be described in a sequential order, such processes, methods, and techniques may be configured to work in alternate orders. In other words, any sequence or order of steps that may be described does not necessarily indicate a requirement that the steps be performed in that order. The steps of processes described herein may be performed in any order practical. Further, some steps may be performed simultaneously. The method of FIG. 2 is also referred to as Dimension-Volume Reduction Iteration (DVRI) technique (or DVRI method) and interchangeably used herein.

[0043] At step **202** of the method of the present disclosure, the one or more hardware processors **104** receive a hyper-dimensional search space, a per iteration sample size, a plurality of associated hyperparameter dimensions, and a termination threshold pertaining to a machine learning (ML) model. The hyper-dimensional search space comprises a plurality of hyperparameters associated with (i) a first hyperparameter dimension, (ii) a second hyperparameter dimension amongst the plurality of associated hyperparameter dimensions. The hyper-dimensional space includes $[h_L^1, h_R^1], \dots, [h_L^d, h_R^d]$, where the subscript L and R denote the left and right of the parameter interval. Other inputs received by the system **100** at step **202** includes a dataset 'D', a per-iteration sample size 'N', an associated hyperparameter dimension 'd', a termination threshold $\alpha=0.01$. Additionally, the system **100** initializes a few parameters associated with the received input and the ML model, such as, but are not limited to, $g_params=$ None, $p_score=0$, $c_score=0$, $g=0$, and the like. The ML model comprises but is not limited to classification model(s) such as linear regression model, and so on. It is to be understood by a person having ordinary skill in the art or person skilled in the art that the above example of ML model as implemented by the system **100** and the method described herein shall not be construed as limiting the scope of the present disclosure. Below Table 1 illustrates hyperparameters and associated dimension(s) pertaining to the machine learning model by way of examples.

TABLE 1

Hyperparameter	Dimension type	parameter range
Penalty	categorical	I1, I2, elasticnet
Tol	numerical(float)	[0, ∞]
C	numerical(float)	[0, 1]
intercept_scaling	numerical(float)	$[-\infty, \infty]$
Solver	categorical	lbfgs, liblinear, newton-cg, newton-cholesky, sag, saga
max_iter	numerical(integer)	[0, ∞]
multi_class	categorical	auto, ovr, multinomial
I1_ratio	numerical(float)	[0, 1]

[0044] The hyperparameter solver as comprised in the ML model and implemented by the system **100** has six categorical elements listed in Table 2, by way of examples.

TABLE 2

Six solvers in the categorical solver hyper-parameter					
lbfgs	liblinear	newton-cg	newton-cholesky	sag	saga

[0045] The hyper-parameter max_iter hyperparameter, which is the discrete type, refers to the maximum number of iterations taken for the solver to converge, and the hyperparameter C, which is the continuous (floating) data type, refers to the inverse of regularization strength: The smaller the value of C, the stronger the regularization. These hyperparameters are crucial in determining the performance of the ML model. For instance, a small max_iter value could cause a premature termination of ML model training process and yield a combination of hyperparameter point and a selection of solver that may affect the computational resource usage of a host machine (e.g., the system **100**) and the performance of the ML model. The objective of hyperparameter optimization is to navigate the hyperparameter space and identify the optimal combination of hyperparameters that yield the best performance of a particular problem. In practice, the optimal hyperparameter is dataset-specific and thus unknown in advance. To get the optimal hyperparameter, one needs to search over large hyper-parameter space (the 8-dimensional hyper-parameter space in LogisticRegression model as an example), which incurs costly computational cost for search, particularly for a large dataset. On the other hand, searching through a small hyper-parameter space can be performed efficiently but produces an underperforming model. The system **100** and the method of the present disclosure efficiently navigate large hyper-parameter space and identify the optimal hyperparameter combination, while substantially reducing computational cost and energy consumption.

[0046] Referring to steps of FIG. 2, at step **204** of the method of the present disclosure, the one or more hardware processors **104** apply, a first reduction technique and a second reduction technique, on the hyper-dimensional search space comprising at least a subset of the plurality of hyperparameters associated with (i) the first hyperparameter dimension, (ii) the second hyperparameter dimension, to obtain a reduced hyper-dimensional search space comprising a set of reduced hyperparameters associated with the second hyperparameter dimension, and a best performance metric of the ML model. In other words, application of the first reduction technique and the second reduction technique on the hyper-dimensional search space results in the reduced

hyper-dimensional search space comprising the set of reduced hyperparameters associated with the second hyperparameter dimension. In an embodiment, the first reduction technique is a Dimension-Reduction Iteration (DRI) technique, as known in the art. In an embodiment, the second reduction technique is a Volume-Reduction Iteration (VRI) technique as known in the art. In an embodiment, the first hyperparameter dimension is a categorical hyperparameter dimension, and the second hyperparameter dimension is a numerical hyperparameter dimension. It is to be understood by a person having ordinary skill in the art or person skilled in the art that such exemplary hyperparameter dimensions shall not be construed as limiting the scope of the present disclosure. The hyper-dimensional search space is based on the per iteration sample size, in one example embodiment.

[0047] At step 206 of the method of the present disclosure, the one or more hardware processors 104 iteratively apply, the first reduction technique and the second reduction technique, on the reduced hyper-dimensional search space comprising the second hyperparameter dimension to obtain an optimal set of hyperparameters. In other words, steps 204 and 206 are performed until a difference of an associated best performance metric between a current iteration and a previous iteration of the ML model is less than or equal to the termination threshold. The performance metric may comprise but is not limited to performance of the ML model, accuracy of the ML model, and so on. In an embodiment, the first reduction technique and the second reduction technique are concurrently applied during each iteration.

[0048] Below pseudo code illustrates the method of FIG. 2 for optimizing hyperparameters of machine learning models by using reduction iteration techniques:

Input: hyper-dimensional space $[h_L^1, h_R^1], \dots, [h_L^d, h_R^d]$, where the subscripts L and R denote the left and right size of the parameter interval.

Input: dataset: D, per-iteration sample size: N, hyper-parameter dimension: d, termination threshold: $\alpha = 0.01$

```

1. initialization: g_params = None, p_score = 0, c_score = 0, g = 0
2. do
3.   c_score = 0
4.   for k = 1 to N # intra-iteration loop
5.     a. for i = 1 to d
6.       b. if  $h_R^i > h_L^i$  then
7.         i. params[i] = random_sample( $[h_L^i, h_R^i]$ )
8.       c. end if
9.       d. end for
10.      e. score = model(params, D)
11.      f. if score > c_score then
12.        i. c_score = score
13.        ii. g_params = params
14.      g. end if
15.    end for
16.  if c_score - p_score <  $\alpha$ :
17.    a. return g_params, g_score
18.  p_score = c_score
19.  for i = 1 to d
20.    a. if g_params[i].type == CAT & g == 0 then
21.      i.  $h_L^i = h_R^i = g\_params[i]$ 
22.      ii. g = 1
23.    b. else
24.      i.  $\Delta_1 = g\_params[i] - h_L^i$ 
25.      ii.  $\Delta_2 = h_R^i - g\_params[i]$ 
26.      iii.  $\Delta = \min(\Delta_1, \Delta_2)$ 
27.      iv.  $h_L^i = g\_params[i] - \Delta/4$ 
28.      v.  $h_R^i = g\_params[i] + \Delta/4$ 
29.    c. end if
30.  end for

```

[0049] In the above pseudo code, lines 1 through 4g indicate taking random sample on each hyper-parameter dimension within the range, wherein a score of the ML model is computed based on the sampled hyperparameter combination. The best score and hyperparameter is stored in memory 102/database 108. Lines 5 through 7 indicate performing a termination condition check. Remaining lines 8 through 9 indicate suppression of categorical hyperparameter dimensions, thereby outputting the range of numerical hyperparameter dimension.

[0050] The above pseudo code may be better understood by way of following description:

[0051] Consider a hyperparameter space (hyper-rectangle) consisting of both categorical and numerical (float and integer) hyperparameter space, per-iteration sample size, the termination condition. The suppression of categorical hyperparameters and the volume reduction of numerical hyperparameters proceeds as described below:

[0052] 1. Take a sample on each hyperparameter range and generate a random hyperparameter point in params in the hyperparameter space.

[0053] 2. Run the ML model against the dataset D, and obtain the score (accuracy or balanced accuracy) of the ML model with respect to the sampled hyperparameter.

[0054] a. point (combination).

[0055] 3. Compare with the score with the value of c_{score} (best one in the iteration)

[0056] 4. If the score is greater than c_{score} , then update c_{score} with the score and the g_params with the values of the params

[0057] 5. If the number of samples is less than the per-iteration sample size, then go back to step 1) and continue processing.

[0058] a. After one-iteration samples, the best score and best hyperparameter combination of the ML model is obtained over the sampled hyperparameter combinations,

[0059] 6. Check the difference between two consecutive iterations. If the difference is less than the threshold if $c_{score} - p_{score} < \alpha$ then return the caller with the best score and best hyperparameter. No further iteration.

[0060] a. Otherwise, suppression and the numerical hyper-parameter volume reduction are as follows.

[0061] 7. Go over each dimension in g_params, check whether the hyperparameter dimension is categorical or numerical.

[0062] 8. If the hyperparameter is categorical (g_params[i].type=CAT and g==0), then assign the value of g_params[i] to both the left- and right-end of that categorical dimension and assign the g value as 1. The categorical dimension is accordingly suppressed.

[0063] 9. If the hyperparameter is numerical, then create a smaller range $[h_L^i, h_R^i]$ centering around the value of g_params[i], which is the best hyperparameter from the iteration.

$$h_L^i = g_params[i] - \Delta/4, h_R^i = g_params[i] + \Delta/4$$

[0064] 10. Return 1) for the next iteration.

[0065] The above steps 204 and 206 are better understood by way of following exemplary description:

[0066] The system and method of the present disclosure consider an example of LogisticRegression as a ML model:

```
Hyperparameter configuration for First Iteration
{"model": "classification", "algorithm":
"LOGISTICREGRESSION",
"hyperparameter": {"solver": ['lbfgs', 'liblinear', 'newton-cg', 'newton-
cholesky', 'sag', 'saga'],
'multi_class': ['auto', 'ovr', 'multinomial'],
'C': ['uniform', (0, 1.0)],
'max_iter': ['randint', (7328, 7597)],
'penalty': ['elasticnet', 'l1', 'l2'],
'l1_ratio': ['uniform', (0.0, 1.0)],
'tol': ['uniform', (1e-6, 1e-5)],
'intercept_scaling': ['uniform', (0.0, 1.0)],
'metric': "accuracy",
'rescale': "normalize",
'sample_size': 20}
```

[0067] After the first iteration, the sampled hyperparameter and score are obtained as follows in Table 3:

TABLE 3

solver	multi_class	C	max_iter	penalty	intercept_scaling	l1_ratio	tol	accuracy
Lbfgs	auto	0.454	7403	elasticnet	0.034	0.4545	0.5e-5	0.922
sag	ovr	0.278	7395	l1	0.933	0.3434	0.6e-5	0.913
Saga	multinomial	0.454	7573	elasticnet	0.534	0.4545	0.5e-5	0.952
Liblinear	ovr	0.478	7339	l1	0.133	0.3434	0.6e-5	0.943
Saga	multinomial	0.454	7758	elasticnet	0.174	0.8088	0.3e-5	0.954
newton-cholesky	ovr	0.278	7433	l2	0.854	0.3434	0.6e-5	0.903
saga	auto	0.454	7690	l2	0.403	0.4545	0.5e-5	0.943
newton-cg	ovr	0.278	7503	l1	0.933	0.6434	0.6e-5	0.923
newton-cholesky	auto	0.454	7389	elasticnet	0.034	0.4545	0.5e-5	0.904
liblinear	auto	0.122	7450	l1	0.743	0.3434	0.53e-5	0.922
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...

[0068] Among the 10 random samples listed above, the best accuracy has been 0.954, and corresponding best sampled hyper-parameter is as in below Table 4 (best hyperparameter combination obtained after the first iteration):

TABLE 4

solver	multi_class	C	max_iter	penalty	intercept_scaling	l1_ratio	tol	accuracy
saga	multinomial	0.454	7758	elasticnet	0.174	0.8088	0.3e-5	0.954

[0069] The dimension reduction iteration method reduces the categorical dimensions and shrinks the ranges of the numerical dimensions simultaneously as follows:

[0070] Suppressing categorical hyper-parameters: From Table 4, the values of solver, multi_class, and penalty dimensions are 'saga', 'multinomial' and 'elasticnet'. Each categorical hyper-parameter dimension is then reduced to a single element. Consequently, no further exploration is required in these categorical dimensions.

[0071] Reducing the search scope for the numerical hyper-parameter space: for every best parameter value in numerical-typed dimension, a much narrower range is computed for subsequent iteration as follows:

$$\Delta = \min(g_params[i] - h_i^L, h_i^R - g_params[i])$$

$$h_i^L = g_params[i] - \Delta/4$$

$$h_i^R = g_params[i] + \Delta/4$$

[0072] The hyperparameter configuration for the second iteration is derived from removing categorical dimensions and reducing the range of numerical dimensions. The value of C dimension for the first iteration is 0.8844, range of the C dimension is calculated as, $\Delta=0.1156$, and the left- and right-end for second iteration are calculated as (0.8555083457, 0.91333050074). The ranges of the numerical hyper-parameter dimensions such as l1_ratio,

intercept_scaling, and C for subsequent iteration are calculated and listed in hyper-parameter configuration in second iteration. As the termination condition $c_{score} - p_{score} = 0.954368 - 0 = 0.954368 > \alpha$, the reduction method performs the second iteration based on the reduced hyperparameter space

specified in the following JavaScript Object Notation (JSON) format for second iteration.

```
Hyperparameter configuration for second iteration
{"model": "classification", "algorithm": "LOGISTICREGRESSION",
"hyperparameter": {"solver": ["saga"],
'multi_class': ["multinomial"],
'C': ["uniform", (0.8555083457, 0.91333050074)],
'max_iter': ["randint", (7328, 7597)]}}
```

-continued

```

'penalty': ['elasticnet'],
'I1_ratio': ['uniform', (0.76102938083, 0.8566176285)],
'tol': ['uniform', (1e-6, 1e-5)],
'intercept_scaling': ['uniform', (
0.13097410199, 0.21829016999)]",
"metric": "accuracy",
"rescale": "normalize",
"sample_size": 20}

```

[0073] After the second iteration of the method of the present disclosure, the best result is given in Table 5. Comparing the accuracy of the second iteration with one obtained from the first one:

$$c_{score} - p_{score} = 0.960490 - 0.954368 = 0.006122 < \alpha = 0.01.$$

[0074] The dimension-reduction iteration method terminates and returns the results as shown in below Table 5 (Optimal hyper-parameter combination after second iteration).

TABLE 5

Hyperparameter dimension	Optimal hyper-parameter combination
penalty	elasticnet
tol	6.242636492935789e-05
C	0.8816418881059275
intercept_scaling	0.1571994484141509
solver	Saga
max_iter	7467
multi_class	Multinomial
I1_ratio	0.799771245720644
Score	0.960490
sample size	20

[0075] In an embodiment, shrinkage (e.g., volume shrinkage, or normalized volume shrinkage and may be interchangeably used herein) of the hyper-dimensional search space between the current iteration and the previous iteration is at the rate of $(x/y)^d$ per-iteration, where d is a dimensionality of the second hyperparameter dimension, and where x and y are integer values. The shrinkage is understood by way of following expressions:

$$\Delta = \min(g_params[i] - h_i^L, h_i^R - g_params[i]),$$

$$h_i^L = g_param[i] - \Delta/4,$$

$$h_i^R = g_params[i] + \Delta/4$$

[0076] The volume shrinkage (also referred to as normalized volume shrinkage and interchangeably used herein) per iteration can be approximately as $(1/4)^d$, in one embodiment of the present disclosure, where $x=1$, and $y=4$, and such values of x and y shall not be construed as limiting the scope of the present disclosure. FIG. 3, with reference to FIGS. 1-2, depicts per-iteration volume shrinkage upon application of the first reduction technique and the second reduction technique on the hyperparameter space, in accordance with an embodiment of the present disclosure.

[0077] In the present disclosure, the system 100 and the method took a small number of samples on the high dimen-

sional hyperparameter space to explore the location of the best optimal hyperparameter combination. Then system 100 makes use of the best hyperparameters in the sampled data to construct smaller hyperparameter space and compares the best performance in the sampled data with the previous one. The process is performed in an iterative fashion, until the performance gain in two consecutive steps is less than the termination threshold.

[0078] FIG. 4, with reference to FIGS. 1 through 3, depicts a graphical representation illustrating samples taking place in two consecutive iterations (left-side figure: dots representing the sampled points in the hyperparameter space, right-side figure: black dots representing the sampled points in the subsequent iteration), in accordance with an embodiment of the present disclosure. The rectangle in the left graph formed by the two best sampled points is used to narrow the hyperparameter space for subsequent iteration. More specifically, in FIG. 4, the objective of iteratively reducing the search space for the best hyperparameter. The method of the present disclosure uses the sampled information to effectively narrow the hyperparameter space for subsequent search in iteration. A small number of samples taken (on the left-side graph) in the preceding iteration, finding the two best sampled point to construct a rectangle, which is much smaller than the sampled hyperparameter space in the preceding hyperparameter space. In the subsequent iteration (on the right-side graph), the samples are taken on the much smaller space.

[0079] The dimension-reduction and the volume-reduction techniques based hyper-parameter optimization method outperforms the as known in the art approaches such as the open-sourced hyper-parameter optimizers (Hyperopt, Optune, Bayesian) in terms of both performance and cost (elapsed time). The performance comparison further solidifies the effectiveness and uniqueness of the dimension-reduction hyperparameter optimization system and method of the present disclosure. FIGS. 5A and 5B, with reference to FIGS. 1 through 4, depict a graphical representation illustrating performance comparison across various approaches. More specifically, FIGS. 5A and 5B depict a graphical representation illustrating performance comparison the method of FIG. 2 (DVRI) and conventional approaches (e.g., Hyperopt®, Optune®, Bayesian optimization, etc.), in accordance with an embodiment of the present disclosure. More specifically, FIG. 5A provides performance comparison of the method of the present disclosure (DVRI method) with the open-source hyper-parameter optimizers. It shows that the method of the present disclosure (DVRI method) performs much better than Hyperopt, Optune, Bayesian hyper-parameter optimizers. FIG. 5B shows that the efficiency of the method of the present disclosure (DVRI method) outperforms both the Hyperopt and Bayesian optimizers and is comparable with Optune hyper-parameter optimizer.

[0080] Below description illustrates the first reduction technique and the second reduction technique implementation versus the conventional approaches.

[0081] 1. Exhaustive Grid search-conventional approach: This is the most widely used method of hyperparameter optimization, and it exhaustively generates candidates from a grid of (discrete) parameter values: for a hyper-parameter grid.

[0082] param_grid=[{'C': [0.02, 0.04, 0.06, 0.08, 0.1, 0.12, 0.14, 0.16, 0.18, 0.20],

[0083] 'solver': ['lbfgs', 'liblinear', 'newton-cg', 'sag', 'saga'],

[0084] 'penalty': ['l1', 'l2', 'elasticnet'],

[0085] 'l1_ratio': [0.02, 0.04, 0.06, 0.08, 0.1, 0.12, 0.14, 0.16, 0.18, 0.20]}}

[0086] The total number of exhaustive grid searches in this case is $10 \times 10 \times 3 \times 5 = 1500$. A shortcoming is that fine-grid exhaustive grid search is extremely costly as it exhaustively examines all grid points, while the coarse grid search is likely to produce suboptimal results.

[0087] 2. Randomized parameter search—conventional approach: This exhaustively generates candidates from a grid of (discrete) parameter values: for a hyper-parameter grid

[0088] param_grid=[{'C': [0.02, 0.04, 0.06, 0.08, 0.1, 0.12, 0.14, 0.16, 0.18, 0.20],

[0089] 'solver': ['lbfgs', 'liblinear', 'newton-cg', 'sag', 'saga'],

[0090] 'penalty': ['l1', 'l2', 'elasticnet'],

[0091] 'l1_ratio': [0.02, 0.04, 0.06, 0.08, 0.1, 0.12, 0.14, 0.16, 0.18, 0.20]}}

[0092] Randomized parameter optimization implements a randomized search over parameters, where the value for each parameter dimension is sampled from a distribution over possible parameter values.

```
{
  'C': scipy.stats.uniform( ),
  'solver': ['lbfgs', 'liblinear', 'newton-cg', 'sag', 'saga'],
  'penalty': ['l1', 'l2', 'elasticnet'],
  'l1_ratio': scipy.stats.uniform( )
}
```

[0093] The value under the C dimension is generated from the uniform distribution. The key difference between the proposed dimension reduction iteration and the randomized search is that the proposed DRI method reduces the dimensionality and volume of hyper-parameter space during processing while the hyper-parameter space is intact for the randomized hyper-parameter search.

[0094] 3. Hyperopt optimization: distributed asynchronous hyperparameter optimization—conventional approach: This approach is primarily based on random search and tree-structure Parzen estimator (TPE). It implements a randomized search over parameters, where the value for each parameter dimension is sampled from a distribution over possible parameter values and evaluates all generated hyperparameters to achieve the best parameter. The key difference between the dimension and volume reduction iteration techniques of the method of the present disclosure and the randomized search is that the dimension and volume reduction iteration techniques of the method of the present disclosure reduces the dimensionality and volume of hyperparameter space during processing while the hyper-parameter space is intact for the randomized hyper-parameter search.

[0095] 4. Bayesian optimization for hyperparameter tuning—conventional approach: This implements Bayesian method, which is built on top of Hyperopt and TPE. It generates the random values for parameter dimensions over predefined distributions over hyper-parameter values and then uses a Bayesian method to identify the best hyper-parameters. The method is com-

putationally expensive. The key difference between the proposed dimension reduction iteration and the randomized search is that the proposed DRI method reduces the dimensionality and volume of hyper-parameter space during processing while the hyper-parameter space is intact for the randomized hyper-parameter search.

[0096] 5. Optuna—conventional approach: Is framework agnostic approach: It employs tree-structure Parzen estimator and random sampling for searching the optimal hyperparameter combination in hyper-parameter space. The dimension and volume reduction iteration techniques of the method of the present disclosure presents a more efficient way than Optuna in the hyper-parameter space reduction.

[0097] The present disclosure provides systems and methods in the unique combination of dimension-reduction and volume-reduction iteration techniques that effectively reduces the computational overhead and energy consumption in machine learning, resulting in significantly enhanced efficiency compared to randomized grid search and hyper-parameter optimizers such as Hyperopt, Optune, and Bayesian optimizers as known in the art.

[0098] The written description describes the subject matter herein to enable any person skilled in the art to make and use the embodiments. The scope of the subject matter embodiments is defined by the claims and may include other modifications that occur to those skilled in the art. Such other modifications are intended to be within the scope of the claims if they have similar elements that do not differ from the literal language of the claims or if they include equivalent elements with insubstantial differences from the literal language of the claims.

[0099] It is to be understood that the scope of the protection is extended to such a program and in addition to a computer-readable means having a message therein; such computer-readable storage means contain program-code means for implementation of one or more steps of the method, when the program runs on a server or mobile device or any suitable programmable device. The hardware device can be any kind of device which can be programmed including e.g., any kind of computer like a server or a personal computer, or the like, or any combination thereof. The device may also include means which could be e.g., hardware means like e.g., an application-specific integrated circuit (ASIC), a field-programmable gate array (FPGA), or a combination of hardware and software means, e.g., an ASIC and an FPGA, or at least one microprocessor and at least one memory with software processing components located therein. Thus, the means can include both hardware means and software means. The method embodiments described herein could be implemented in hardware and software. The device may also include software means. Alternatively, the embodiments may be implemented on different hardware devices, e.g., using a plurality of CPUs.

[0100] The embodiments herein can comprise hardware and software elements. The embodiments that are implemented in software include but are not limited to, firmware, resident software, microcode, etc. The functions performed by various components described herein may be implemented in other components or combinations of other components. For the purposes of this description, a computer-usable or computer readable medium can be any apparatus that can comprise, store, communicate, propagate, or trans-

port the program for use by or in connection with the instruction execution system, apparatus, or device.

[0101] The illustrated steps are set out to explain the exemplary embodiments shown, and it should be anticipated that ongoing technological development will change the manner in which particular functions are performed. These examples are presented herein for purposes of illustration, and not limitation. Further, the boundaries of the functional building blocks have been arbitrarily defined herein for the convenience of the description. Alternative boundaries can be defined so long as the specified functions and relationships thereof are appropriately performed. Alternatives (including equivalents, extensions, variations, deviations, etc., of those described herein) will be apparent to persons skilled in the relevant art(s) based on the teachings contained herein. Such alternatives fall within the scope of the disclosed embodiments. Also, the words “comprising,” “having,” “containing,” and “including,” and other similar forms are intended to be equivalent in meaning and be open ended in that an item or items following any one of these words is not meant to be an exhaustive listing of such item or items, or meant to be limited to only the listed item or items. It must also be noted that as used herein and in the appended claims, the singular forms “a,” “an,” and “the” include plural references unless the context clearly dictates otherwise.

[0102] Furthermore, one or more computer-readable storage media may be utilized in implementing embodiments consistent with the present disclosure. A computer-readable storage medium refers to any type of physical memory on which information or data readable by a processor may be stored. Thus, a computer-readable storage medium may store instructions for execution by one or more processors, including instructions for causing the processor(s) to perform steps or stages consistent with the embodiments described herein. The term “computer-readable medium” should be understood to include tangible items and exclude carrier waves and transient signals, i.e., be non-transitory. Examples include random access memory (RAM), read-only memory (ROM), volatile memory, nonvolatile memory, hard drives, CD ROMs, DVDs, flash drives, disks, and any other known physical storage media.

[0103] It is intended that the disclosure and examples be considered as exemplary only, with a true scope of disclosed embodiments being indicated by the following claims.

What is claimed is:

1. A processor implemented method, comprising:

receiving, via one or more hardware processors, a hyper-dimensional search space, a per iteration sample size, one or more associated hyperparameter dimensions, and a termination threshold pertaining to a machine learning (ML) model, wherein the hyper-dimensional search space comprises a plurality of hyperparameters associated with (i) a first hyperparameter dimension, and (ii) a second hyperparameter dimension;

applying, a first reduction technique and a second reduction technique via the one or more hardware processors, on the hyper-dimensional search space comprising at least a subset of the plurality of hyperparameters associated with (i) the first hyperparameter dimension, (ii) the second hyperparameter dimension, to obtain a reduced hyper-dimensional search space comprising a set of reduced hyperparameters associated with the second hyperparameter dimension, and a best performance

metric of the ML model, wherein the hyper-dimensional search space is based on the per iteration sample size; and

iteratively applying, the first reduction technique and the second reduction technique via the one or more hardware processors, on the reduced hyper-dimensional search space comprising the second hyperparameter dimension to obtain an optimal set of hyperparameters, until a difference of an associated best performance metric between a current iteration and a previous iteration of the ML model is less than or equal to the termination threshold.

2. The processor implemented method of claim 1, wherein shrinkage of the hyper-dimensional search space between the current iteration and the previous iteration is at the rate of $(x/y)^d$ per-iteration, where d is a dimensionality of the second hyperparameter dimension, and x and y are integer values.

3. The processor implemented method of claim 1, wherein the first reduction technique is a Dimension-Reduction Iteration (DRI) technique, and wherein the second reduction technique is a Volume-Reduction Iteration (VRI) technique.

4. The processor implemented method of claim 1, wherein the first reduction technique and the second reduction technique are concurrently applied during each iteration.

5. The processor implemented method of claim 1, wherein the first hyperparameter dimension is a categorical hyperparameter dimension, and the second hyperparameter dimension is a numerical hyperparameter dimension.

6. A system, comprising:

a memory storing instructions;

one or more communication interfaces; and

one or more hardware processors coupled to the memory via the one or more communication interfaces, wherein the one or more hardware processors are configured by the instructions to:

receive a hyper-dimensional search space, a per iteration sample size, one or more associated hyperparameter dimensions, and a termination threshold pertaining to a machine learning (ML) model, wherein the hyper-dimensional search space comprises a plurality of hyperparameters associated with (i) a first hyperparameter dimension, and (ii) a second hyperparameter dimension;

apply, a first reduction technique and a second reduction technique, on the hyper-dimensional search space comprising at least a subset of the plurality of hyperparameters associated with (i) the first hyperparameter dimension, (ii) the second hyperparameter dimension, to obtain a reduced hyper-dimensional search space comprising a set of reduced hyperparameters associated with the second hyperparameter dimension, and a best performance metric of the ML model, wherein the hyper-dimensional search space is based on the per iteration sample size; and

iteratively apply, the first reduction technique and the second reduction technique, on the reduced hyper-dimensional search space comprising the second hyperparameter dimension to obtain an optimal set of hyperparameters,

until a difference of an associated best performance metric between a current iteration and a previous iteration of the ML model is less than or equal to the termination threshold.

7. The system of claim 6, wherein shrinkage of the hyper-dimensional search space between the current iteration and the previous iteration is at the rate of $(x/y)^d$ a per-iteration, where d is a dimensionality of the second hyperparameter dimension, and x and y are integer values.

8. The system of claim 6, wherein the first reduction technique is a Dimension-Reduction Iteration (DRI) technique, and wherein the second reduction technique is a Volume-Reduction Iteration (VRI) technique.

9. The system of claim 6, wherein the first reduction technique and the second reduction technique are concurrently applied during each iteration.

10. The system of claim 6, wherein the first hyperparameter dimension is a categorical hyperparameter dimension, and the second hyperparameter dimension is a numerical hyperparameter dimension.

11. One or more non-transitory machine-readable information storage mediums comprising one or more instructions which when executed by one or more hardware processors cause:

receiving a hyper-dimensional search space, a per iteration sample size, one or more associated hyperparameter dimensions, and a termination threshold pertaining to a machine learning (ML) model, wherein the hyper-dimensional search space comprises a plurality of hyperparameters associated with (i) a first hyperparameter dimension, (ii) a second hyperparameter dimension;

applying a first reduction technique and a second reduction technique on the hyper-dimensional search space comprising at least a subset of the plurality of hyperparameters associated with (i) the first hyperparameter dimension, (ii) the second hyperparameter dimension, to obtain a reduced hyper-dimensional search space comprising a set of reduced hyperparameters associ-

ated with the second hyperparameter dimension, and a best performance metric of the ML model, wherein the hyper-dimensional search space is based on the per iteration sample size; and

iteratively applying the first reduction technique and the second reduction technique on the reduced hyper-dimensional search space comprising the second hyperparameter dimension to obtain an optimal set of hyperparameters,

until a difference of an associated best performance metric between a current iteration and a previous iteration of the ML model is less than or equal to the termination threshold.

12. The one or more non-transitory machine-readable information storage mediums of claim 11, wherein shrinkage of the hyper-dimensional search space between the current iteration and the previous iteration is at the rate of $(x/y)^d$ a per-iteration, where d is a dimensionality of the second hyperparameter dimension, and x and y are integer values.

13. The one or more non-transitory machine-readable information storage mediums of claim 11, wherein the first reduction technique is a Dimension-Reduction Iteration (DRI) technique, and wherein the second reduction technique is a Volume-Reduction Iteration (VRI) technique.

14. The one or more non-transitory machine-readable information storage mediums of claim 11, wherein the first reduction technique and the second reduction technique are concurrently applied during each iteration.

15. The one or more non-transitory machine-readable information storage mediums of claim 11, wherein the first hyperparameter dimension is a categorical hyperparameter dimension, and the second hyperparameter dimension is a numerical hyperparameter dimension.

* * * * *